# Heterogeneous Graph Learning for Article Recommender System

DS300.Q11 - Final Project

**Presented by:**

T. Nhat[1] L. M. Nhut[1] N. H. Phat[1]

**Instructor**: Master. Huynh Van Tin[2]

[1]Faculty of Computer Science
University of Information Technology
[2]Faculty of Information Science and Technology
University of Information Technology

DS300 Final Project Report

# Table of Contents


1. Project Overview & Methodology
2. Dataset Construction and EDA
3. Motivation: Graph Expansion Strategy
4. State-of-the-Art (SOTA) Baselines
5. Proposed Method: Heterogeneous Graph Learning
6. Experimental Results & Innovations
7. Conclusion & Future Works

# Table of Contents


1. Project Overview & Methodology
2. Dataset Construction and EDA
3. Motivation: Graph Expansion Strategy
4. State-of-the-Art (SOTA) Baselines
5. Proposed Method: Heterogeneous Graph Learning
6. Experimental Results & Innovations
7. Conclusion & Future Works

# Motivation: Why News Recommendation?

In the digital era, news platforms face **Information Overload**:

- **User Experience:** Navigating thousands of daily articles is overwhelming.
- **Retention:** Personalized content increases user engagement and platform stickiness.
- **Monetization:** Better engagement leads to higher ad revenue and subscriptions.

**Target:** Build a system that understands both *Content semantics* and *User behavior*.

# Research Objectives

Our goal is to develop a recommendation engine for Vietnamese news:

- **Graph Learning:** Leverage GNNs to model complex user-article interactions.
- **Multimodal Fusion:** Incorporate Social (replies) signals.
- **Vietnamese NLP:** Optimize PhoBERT embeddings for news semantics.

# Problem Statement

**Challenge:** How to recommend relevant news articles to users?

**Key Challenges:**

1. **Cold Start:** New users/articles have no interaction history
2. **Sparse Data:** Most users read very few articles (99.9% sparsity)
3. **Vietnamese Language:** Limited NLP resources for Vietnamese text
4. **Interaction Limitation:** Don't have access different kind of interactions.

**Our Approach:** Graph-based Contrastive Learning with PhoBERT embeddings

# Table of Contents


1. Project Overview & Methodology
2. Dataset Construction and EDA
3. Motivation: Graph Expansion Strategy
4. State-of-the-Art (SOTA) Baselines
5. Proposed Method: Heterogeneous Graph Learning
6. Experimental Results & Innovations
7. Conclusion & Future Works

# VnExpress Data Crawling

**Data Source:** VnExpress.net (largest Vietnamese news portal)

**Categories Crawled:**

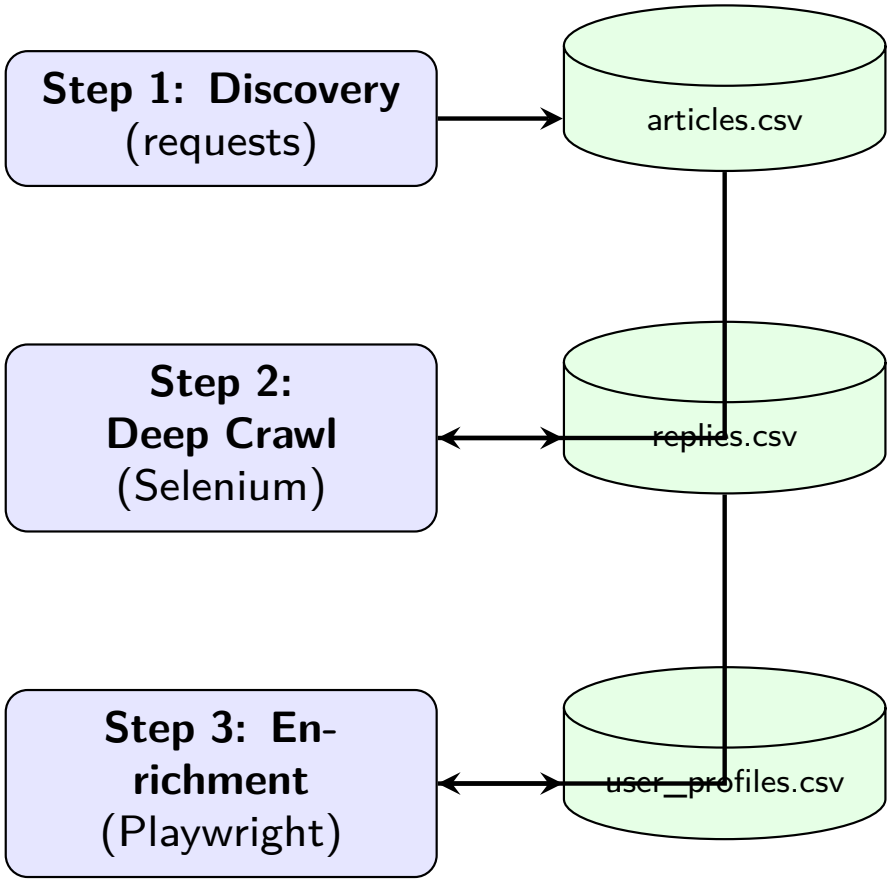
- 'Thế giới'
- 'Giáo dục'
- 'Thể thao'
- 'Thời sự'
- 'Kinh doanh'
- 'Khoa học công nghệ'

**Collected Data:**

- Articles: title, content, author, publish date
- Metadata: category, tags (per article)
- Comments: user ID, comment text, reactions, replies
- User profiles: user ID, username, join date

# Data Collection Pipeline: 3-Step Workflow

Our crawler implements a resilient 3-stage architecture to handle high-volume news data:




Parallel scanning of category pages to find new URLs.

Parallel browser workers to extract deep comment threads & nested replies.

Scraping full history for all unique active commenters.

# Pipeline Orchestration & Efficiency

- **Parallelism:** `Step 2` (Selenium) is resource-intensive. We split the workload across $N$ parallel browser instances using `multiprocessing`.
- **Resiliency:**
  - **Caching:** Avoids redundant HTTP requests for static content.
  - **Checkpointing:** Atomic writes to CSV allow resuming from the last article in case of failure.
  - **Headless Execution:** Minimizes resource overhead in production environments.
- **Tool Stack:**
  - **Requests:** High-speed metadata extraction.
  - **Selenium:** Handling complex JS (Load More buttons).
  - **Playwright:** Modern, fast engine for user history scraping.

# Dataset Overview: High-Level Statistics

**Raw Behavioral Data:**

- **3,685** Articles
- **42,050** Comments
- **6** News Categories
- **19,842** User Profiles

**Graph Interaction Set (G1-G3):**

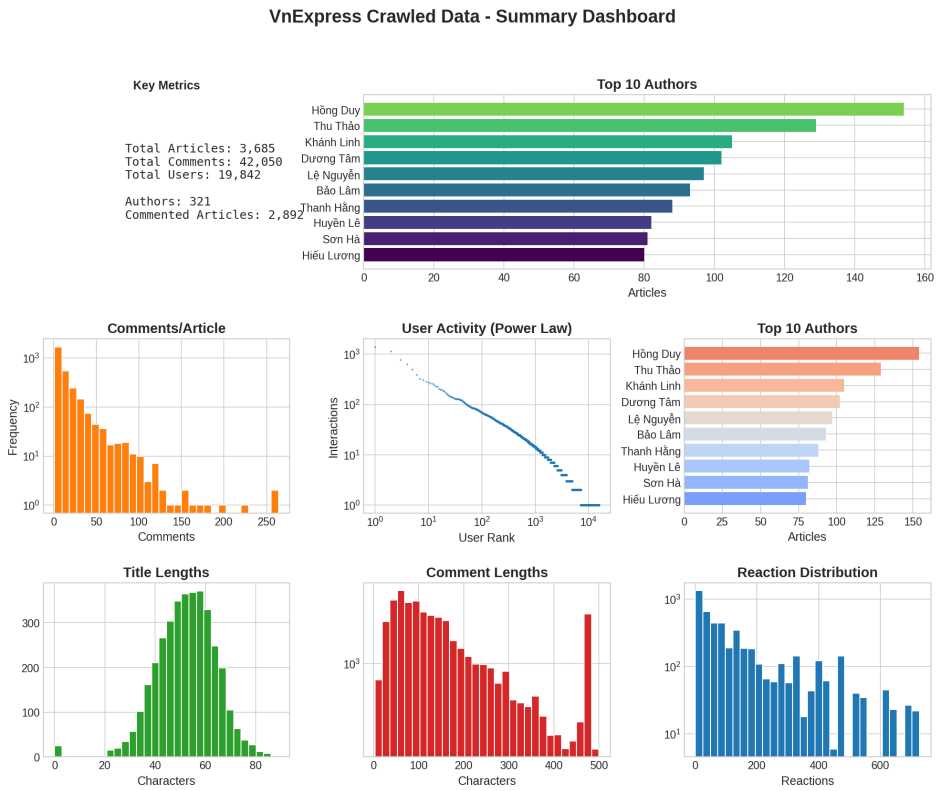
- **13,117** Unique Users
- **3,645** Active Articles
- **38,505** Total Interactions
- **Sparsity: 99.919%**

# Dataset Overview (VnExpress)

**Statistics:**

- **Users:** 13,117
- **Articles:** 3,645
- **Interactions:** 38,505
- **Sparsity:** 99.92%

The dataset is highly sparse, typical of news domains where users read only a tiny fraction of daily content.



# Data Sample: Interaction & Content

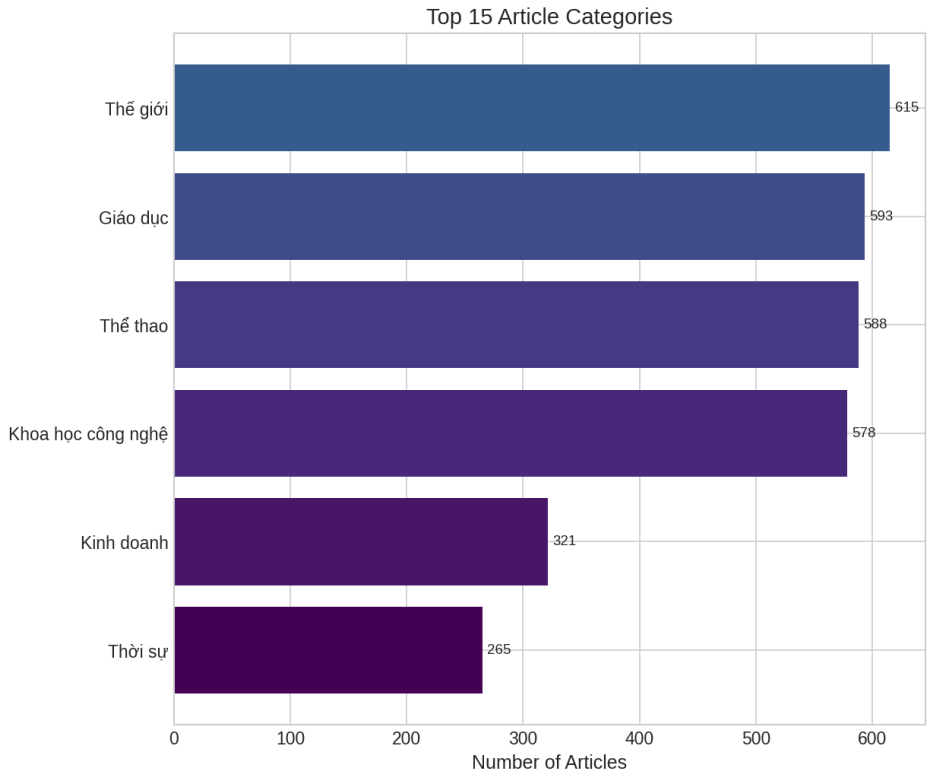
1. **Content Sample (Article)**
*"Chinh phục loạt 'ông lớn' công nghệ sau hơn 500 lần bị từ chối"*
→ **Tags:** [ Meta, Amazon, Apple, Tesla, kĩ sư công nghệ ]

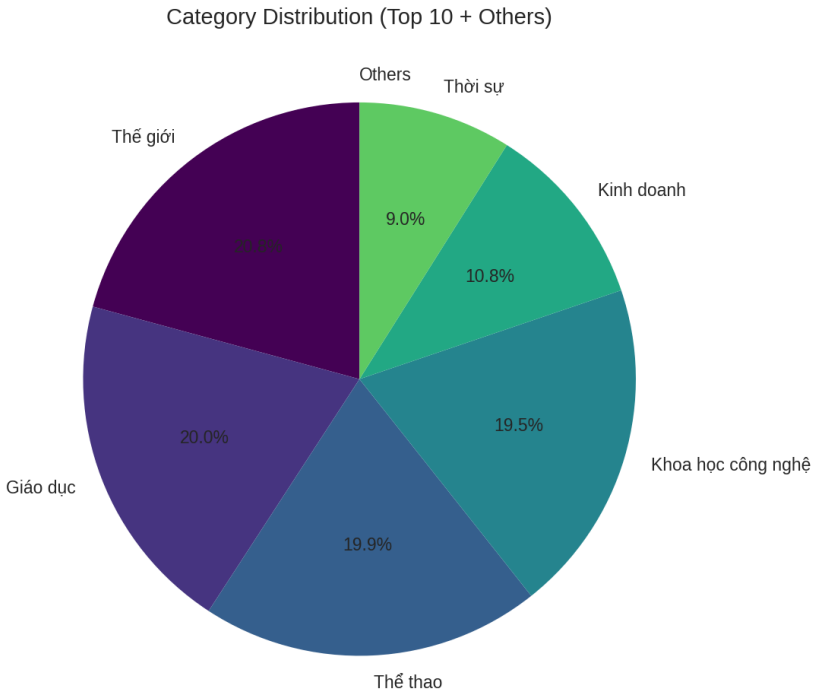
2. **Social Sample (Comment/Reply)**
   - **User A:** "Nếu giỏi thì cố gắng mua lại mấy công ty này..."
   - **User B:** (Reply to A) "Mua lại Meta chắc cần nghìn tỷ USD..."
   ⇒ Social Connection detected

## Feature Mapping

- **Article** → PhoBERT Embedding ($d = 64$) → Node Feature.
- **User** → Interaction aggregation → Node Embedding.
- **Reply** → User-User Edge in the Heterogeneous Graph.

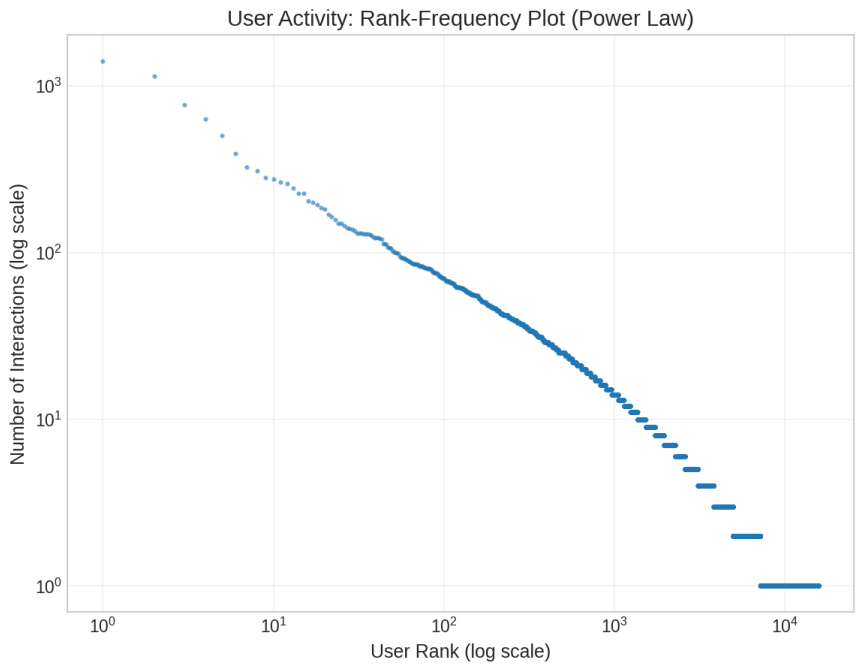
# Category Distribution (VnExpress)

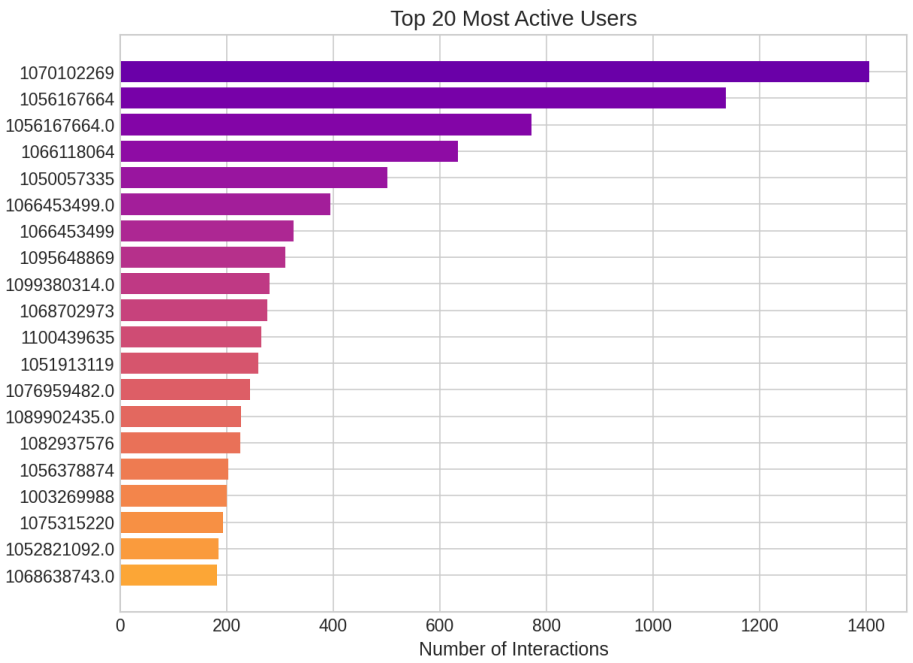



- **Thế giới** and **Thời sự** are the dominant categories.
- Skewed distribution reflects editorial priorities.

# User Engagement Patterns (1/2)

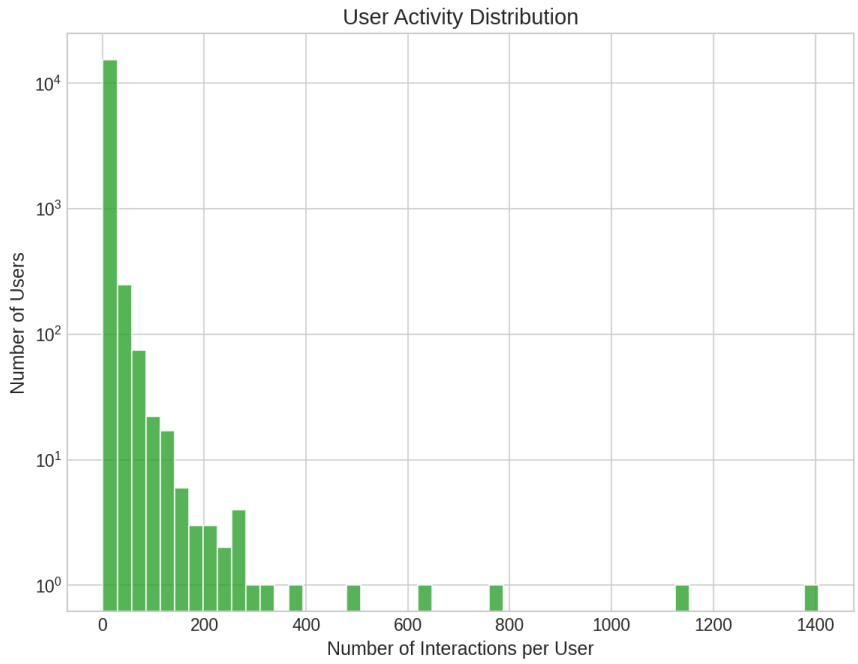


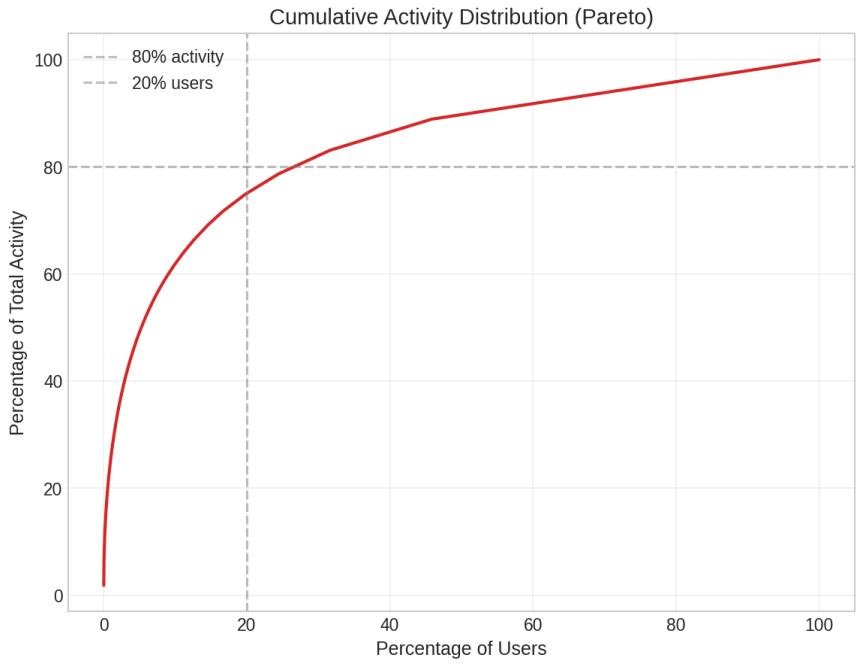




## Distribution Analysis

Article popularity follows Zipf's law. Top 10 users account for 15% of all interactions.

# User Engagement Patterns (2/2)

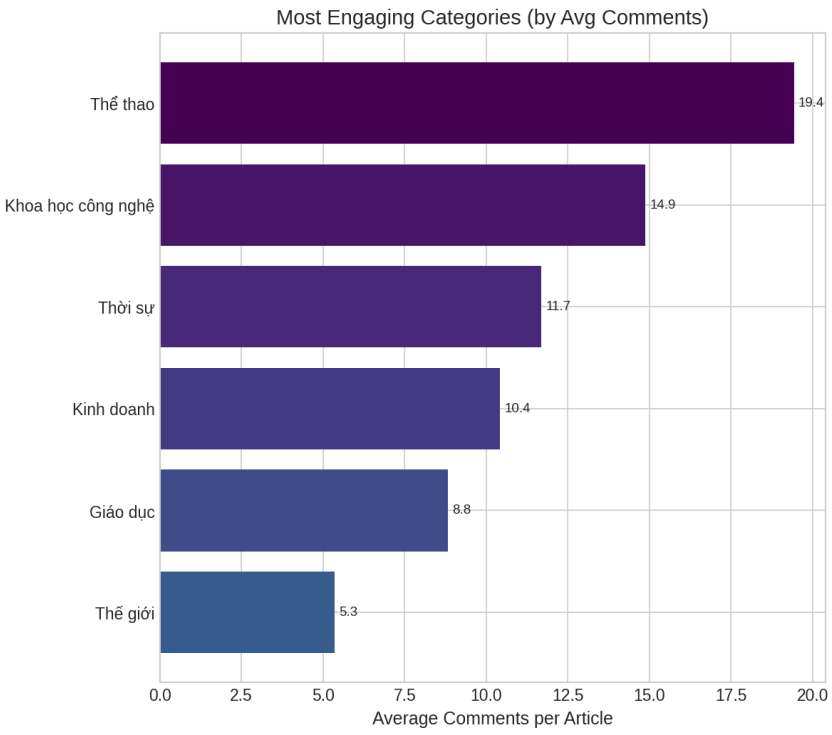


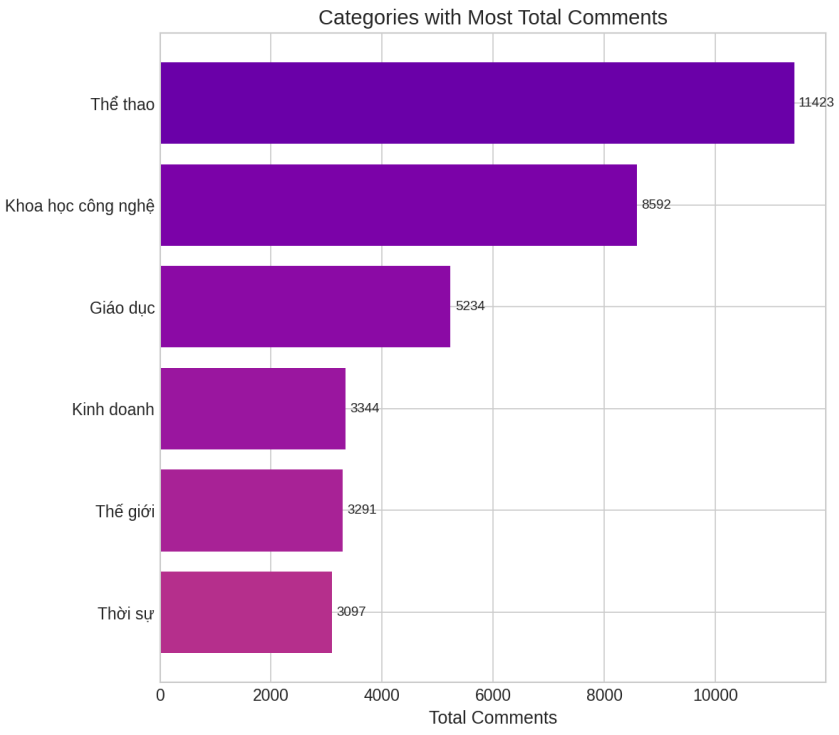




## Power Law Observation

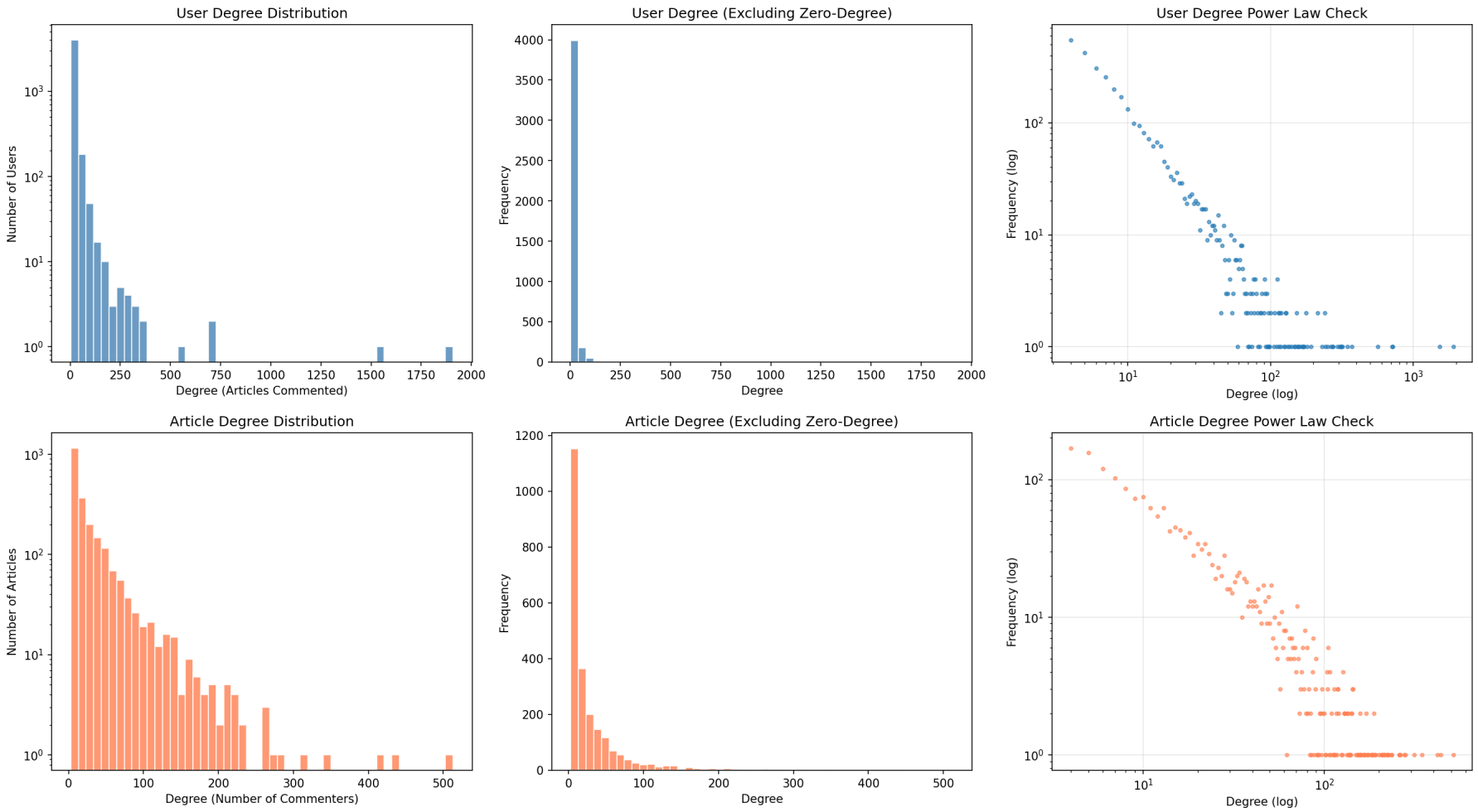
Long-tail distribution: 80% of users have fewer than 10 interactions.

# Engagement by Category



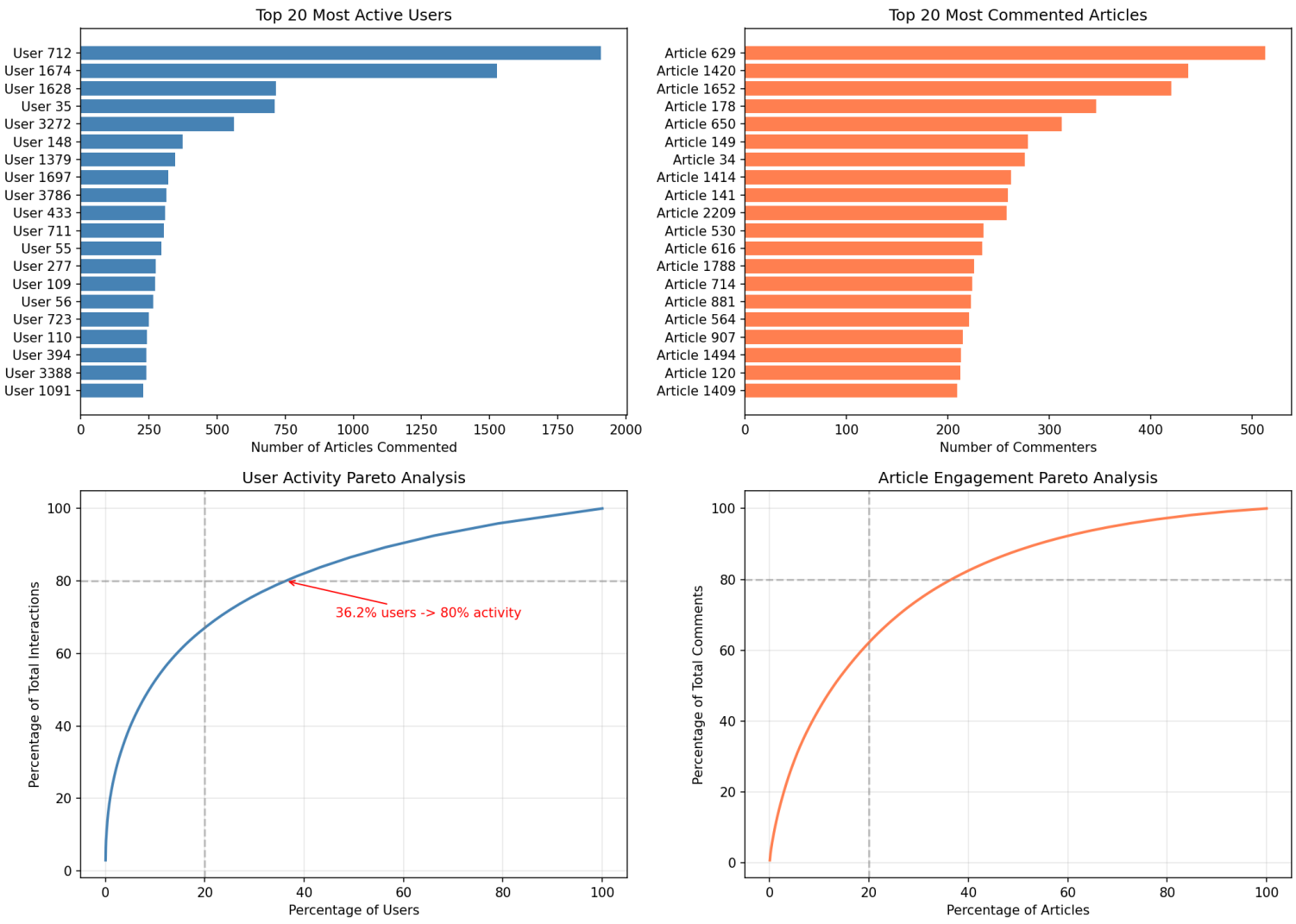



# Graph Degree Distribution



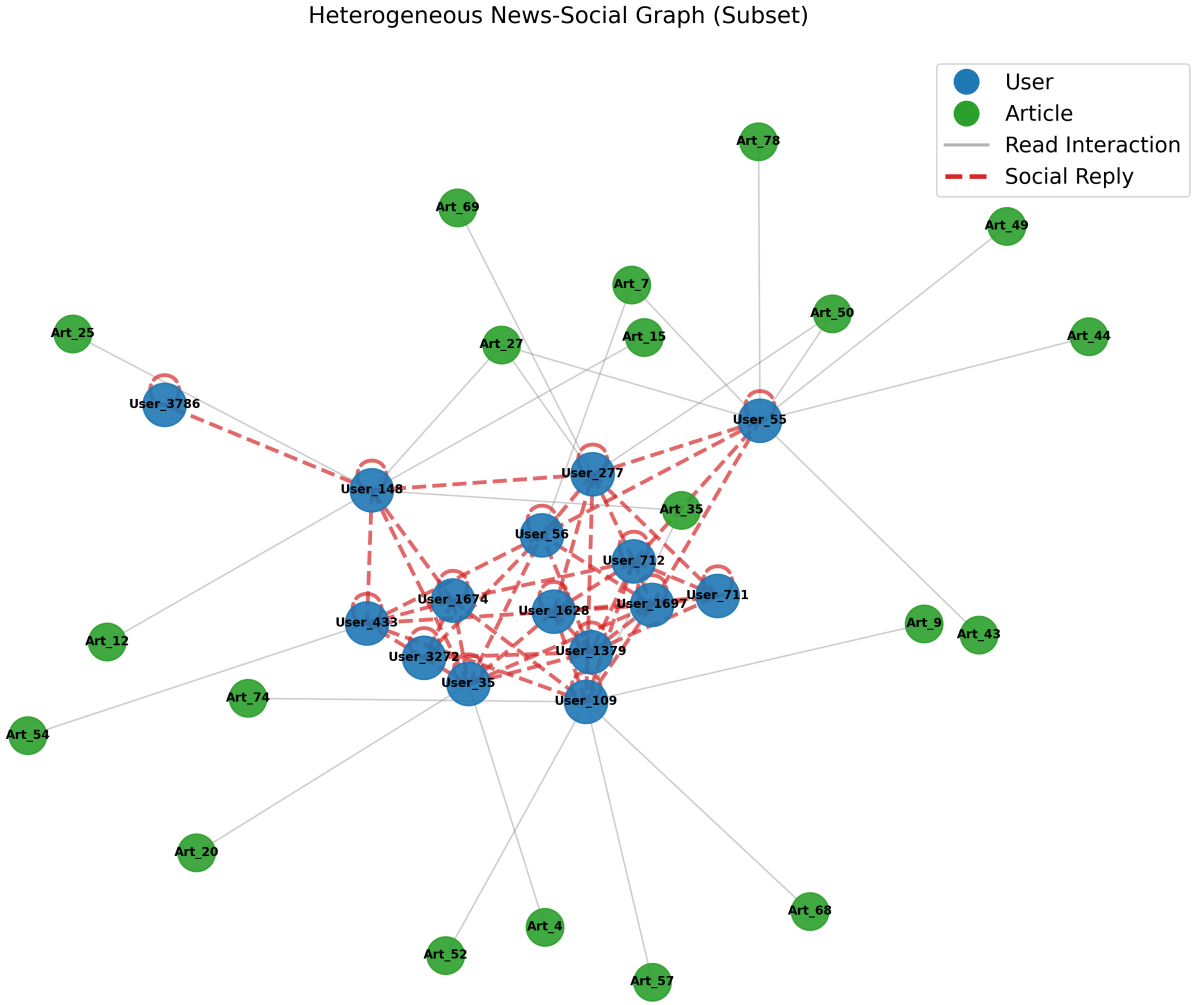

- User degrees peak at very low values (high cold-start risk).
- Article degrees are more distributed but still sparse.

# Graph Connectivity & Density



# Graph Topology

Visualizing the User-Article-Comment bipartite graph structure.



# Table of Contents


1. Project Overview & Methodology
2. Dataset Construction and EDA
3. Motivation: Graph Expansion Strategy
4. State-of-the-Art (SOTA) Baselines
5. Proposed Method: Heterogeneous Graph Learning
6. Experimental Results & Innovations
7. Conclusion & Future Works

# The Sparsity Challenge

**Problem:** Interactions are extremely sproparse (99.919%), making it hard for models to learn.

## Sparsity Calculation

$$\text{Sparsity} = 1 - \frac{38,505 \text{ (Interactions)}}{13,117 \text{ (Users)} \times 3,645 \text{ (Articles)}} = \mathbf{99.919\%}$$

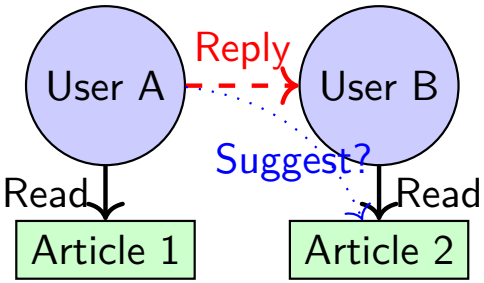
# Solution: Expanding the Graph Structure

We progressively enrich the graph to alleviate sparsity. We define **3 Graph Tiers** for experimentation:

- **Graph G1 (Bipartite Baseline):**
  - Nodes: Users, Articles.
  - Edges: *Comment*.
  - *Standard CF setting.*
- **Graph G2 (Social Expansion - "The Increase"):**
  - **+ User-User Edges:** Created via "Reply" interactions.
  - *Hypothesis: Users who converse share interests.*
- **Graph G3 (Semantic Expansion):**
  - **+ Category Nodes:** Users connect to Categories they read.
  - *Hypothesis: High-level hubs reduce path length.*

# Core Innovation: Social Fusion (Graph G2)

**Idea:** Use the "Reply Network" to bridge gaps between users.




**The Logic:**

- **Direct:** User A likes Article 1.
- **Social:** User A replies to User B $\rightarrow$ *Shared Interest*.
- **Inference:** User A is likely to enjoy Article 2 (read by B).

## Impact on Sparsity

By adding User-User edges (G2), the GNN can propagate "interests" even if User A has only read 1 article.

# Table of Contents


1. Project Overview & Methodology
2. Dataset Construction and EDA
3. Motivation: Graph Expansion Strategy
4. State-of-the-Art (SOTA) Baselines
5. Proposed Method: Heterogeneous Graph Learning
6. Experimental Results & Innovations
7. Conclusion & Future Works

# Current SOTA Approaches

We benchmark against three main categories of modern Recommender Systems:

1. **Graph Neural Networks (GNNs):**
   - **LightGCL (2023):** Efficient Global Context Learning via SVD for better structural alignment.
   - **XSimGCL (2023):** Extreme Simple Graph Contrastive Learning using noise perturbation instead of graph augmentation.
   - **LightGCN (2020):** Simplified GCN by removing non-linearities. Current industry standard for efficiency.
2. **Contrastive Learning (CL):**
   - **SimGCL (2022):** Augments graph structure with noise to learn robust representations. Handles sparsity better than pure GNNs.
3. **Content-Based & Hybrid:**
   - **PhoBERT:** State-of-the-art BERT model for Vietnamese, capturing deep semantic text features.

# State-of-the-Art Baselines: Contrastive Learning

## 1. LightGCL (Global Context)

**Problem:** Standard GNNs only capture local neighbors.
**Solution:** Learn **Global Context** via SVD ($G = U_k \Sigma_k V_k^T$).
**Why News?** Breaks "Info Cocoons" by recommending distinct but related topics.

## 2. XSimGCL (Extreme Simple)

**Problem:** Graph augmentation (drop edge) is slow.
**Solution:** Add random **Noise** ($\Delta$) to each embeddings layers: $e' = e + \Delta$.
**Benefit:** Fast (34s/epoch) & Robust regularization.

*Both methods outperform traditional GCNs by improving representations.*

# HGRec: Heterogeneous Graph Neural Network

**Core Idea:** Real-world graphs have multiple node/edge types (e.g., User-Reply-User vs. User-Comment-Article). Standard GNNs ignore this distinction [1].

## 1. Meta-path Aggregation

Separately aggregates neighbors for each relation type:

$$h_u^{\Phi} = \text{AGG}\{h_v | v \in \mathcal{N}_u^{\Phi}\}$$

*Example:* Captures "Social" signals separately from "Interest" signals.

## 2. Semantic Fusion

Combines type-specific embeddings using Attention:

$$h_u^{Final} = \sum_{\Phi} \beta_{\Phi} \cdot h_u^{\Phi}$$

*Benefit:* Learns which relation is more important (e.g., Social > Interest for inactive users).

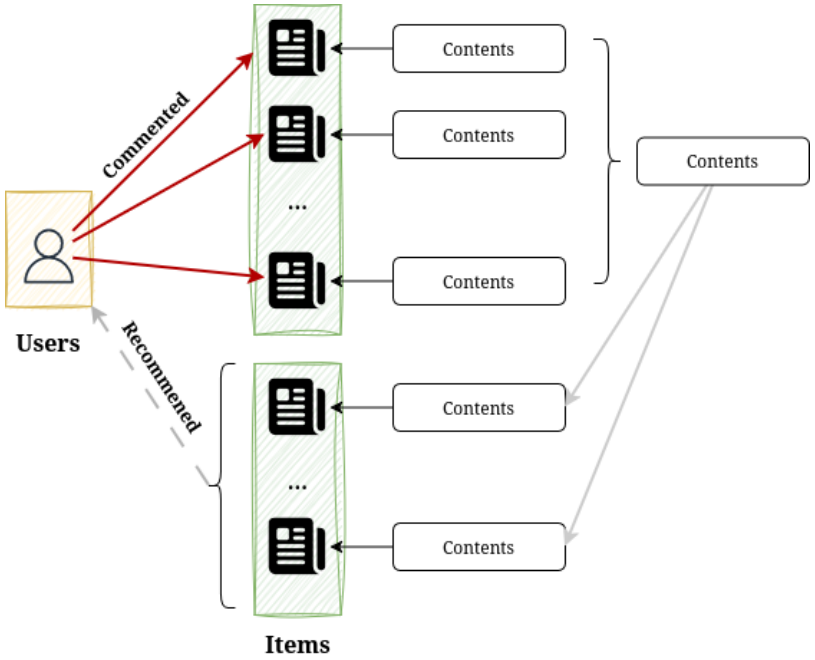
**Significance:** This mechanism forms the backbone of our **HGN** and **HCL** models.

# Approach 1: Content-Based Filtering (CB)

**Concept:** Recommend items similar to what the user liked in the past.

## Process Flow

- **Input:** Title, Abstract.
- **Method:** PhoBERT $\rightarrow$ Dense Vectors.
- **Metric:** Cosine Similarity.
- **Output:** Top-K semantically similar items.

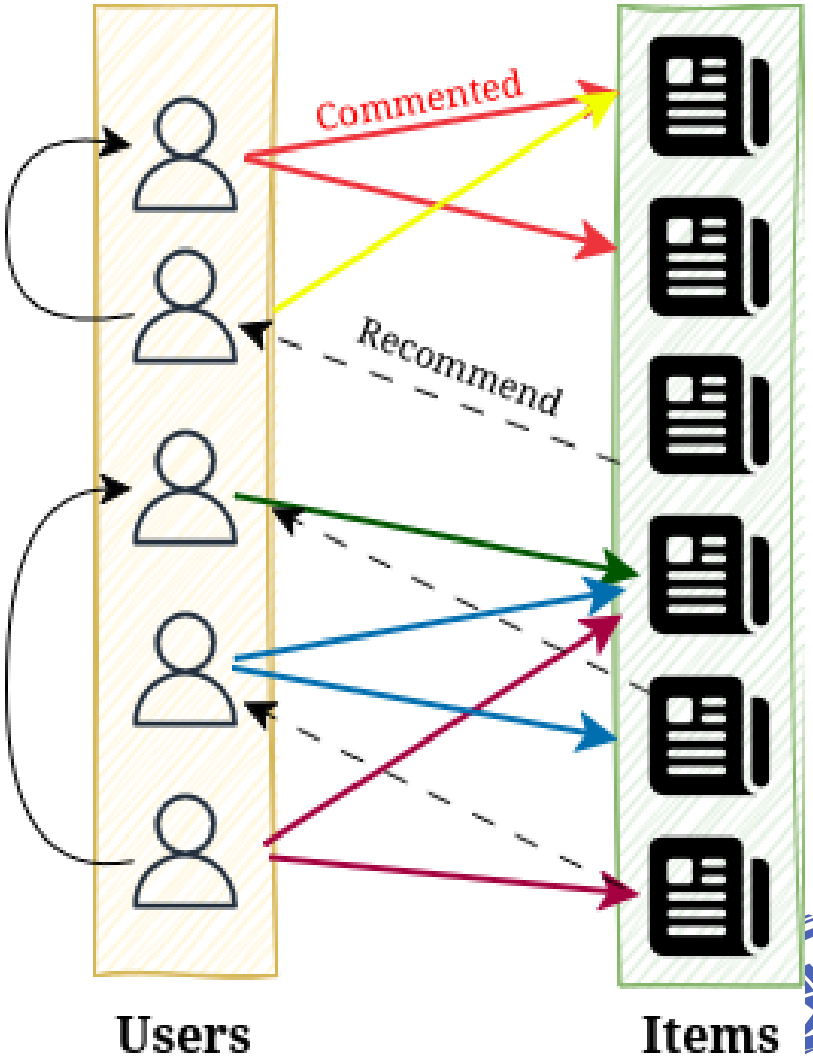


# Approach 2.1: User-User Collaborative Filtering

**Concept:** Leverage "Crowd Wisdom" - "Users like you also liked..."

## Mechanism

1. Identify similar users based on interaction history (e.g., Cosine Similarity).
2. Aggregate items consumed by these neighbors.
3. Recommend items unseen by the target user.

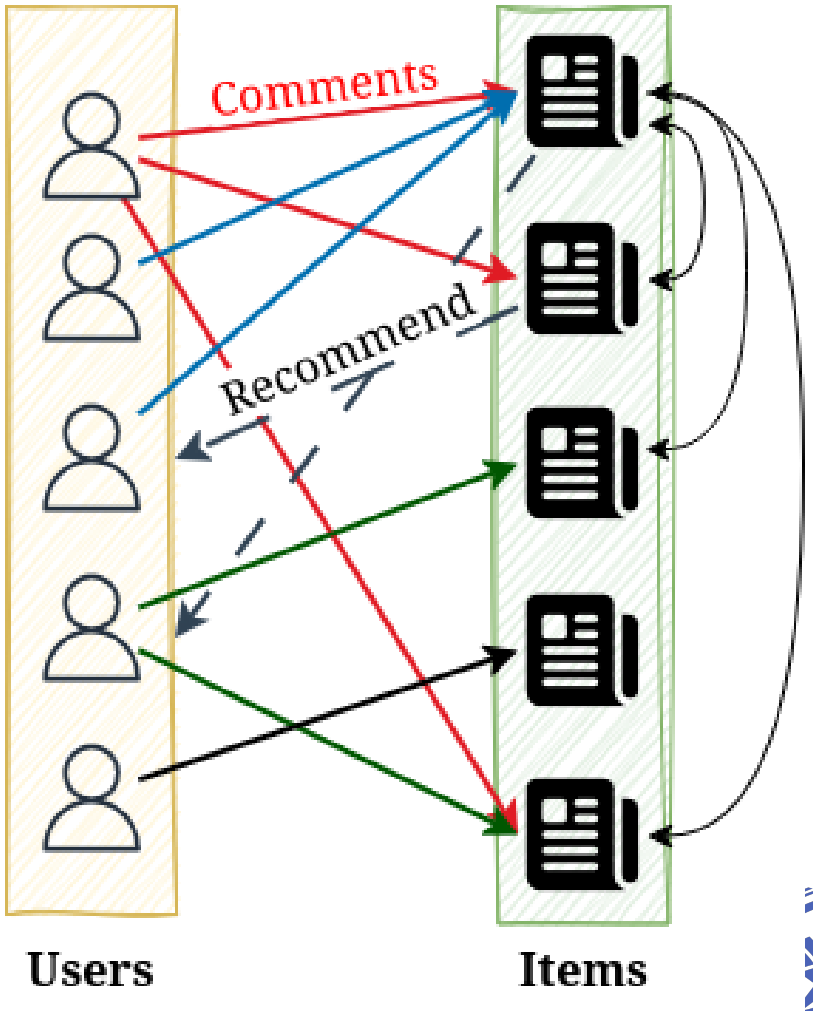


# Approach 2.2: Item-Item Collaborative Filtering

**Concept:** "Users who bought X also bought Y"

## Mechanism

1. Calculate similarity between items based on co-occurrence in user histories.
2. If a user likes Item A, recommend Item B (where A and B are similar).
3. More stable than User-User as the item set is strictly static.



# Table of Contents


1. Project Overview & Methodology
2. Dataset Construction and EDA
3. Motivation: Graph Expansion Strategy
4. State-of-the-Art (SOTA) Baselines
5. Proposed Method: Heterogeneous Graph Learning
6. Experimental Results & Innovations
7. Conclusion & Future Works

# Inspiration & Theoretical Foundation

**Where did this idea come from?** Our architecture is an adaptation of the **HGRec** framework [1], optimized with LightGCN **he2020lightgcn** mechanics.

## 1. HGRec (Heterogeneous Graph Neural Network)

**Heterogeneous GNN for Recommendation**

- **Key Idea:** Users and items have multiple types of relationships (e.g., Comment vs Reply).
- **Mechanism:** Use relation-specific aggregation steps.

## 2. LightGCN

**Simplified GCN for Recommendation**

- **Key Idea:** Non-linearities (ReLU) are effectively redundant for collaborative filtering.
- **Mechanism:** Linear propagation + Layer averaging.

# Motivation: Why a Heterogeneous GNN?

**The Problem with Standard GNNs (LightGCN):**

- They treat the graph as **Homogeneous**: All nodes are just "IDs", all edges are just "links".
- **Failure Case:** A "Reply" (social trust) is treated identically to a "Comment" (explicit interest).

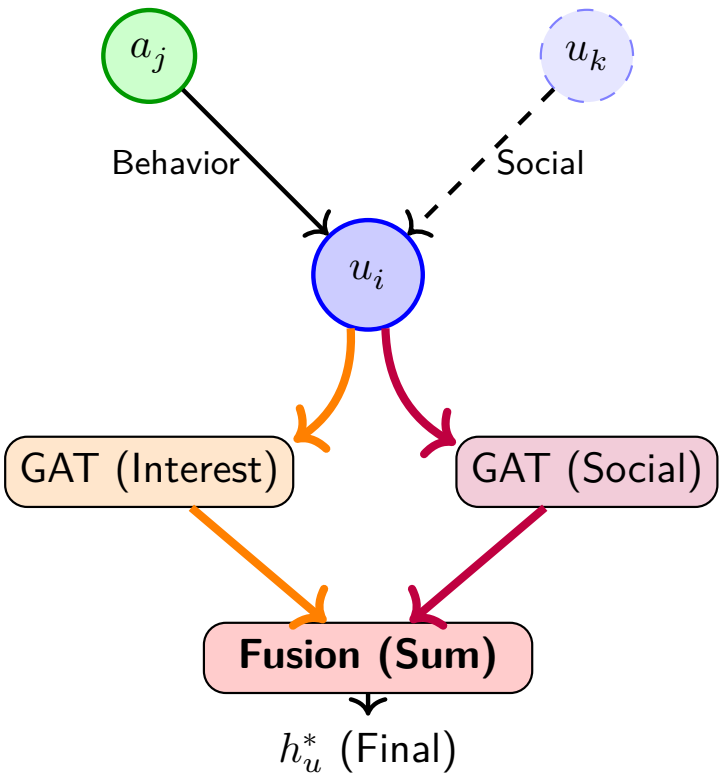
**The VnExpress Reality:**

- **Multiple Node Types:** Users $\neq$ Items.
- **Multiple Edge Types:**
  - `User-Comment-Item`: Explicit Interest.
  - `User-Reply-User`: Social Influence / Trust.

## The HGN Solution

We need an architecture that aggregates **Social Signals** separately from **Content Signals** before fusing them.

# HGN: Multi-Aspect Heterogeneous GNN




**Core Innovation: Dual-path Propagation**

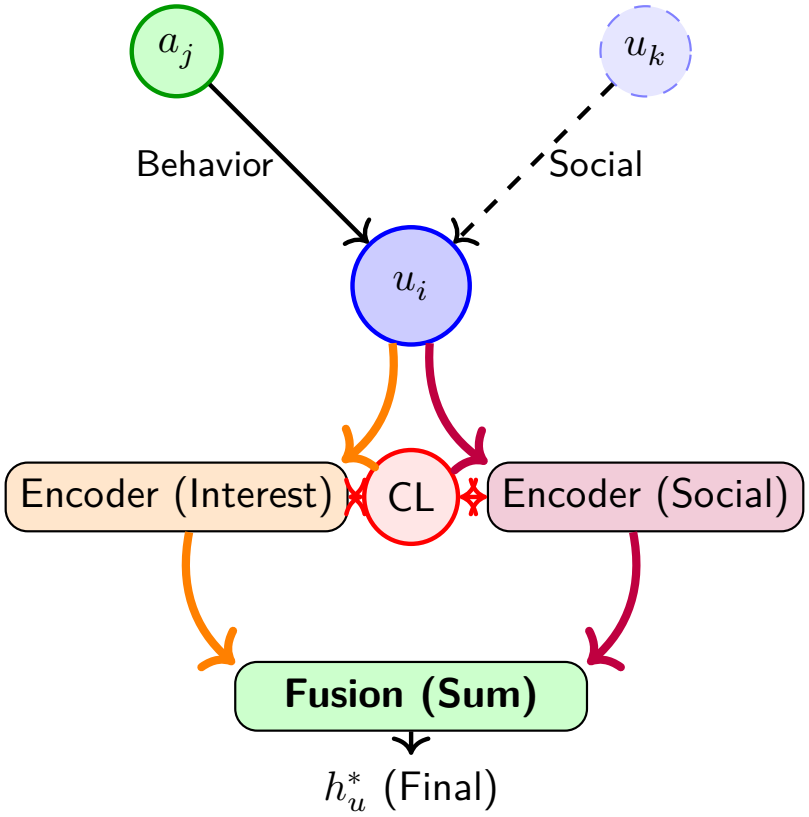
- **Path 1 (Interest):** history ($\mathcal{G}_{u-a}$) via **GAT**.
- **Path 2 (Social):** friends ($\mathcal{G}_{u-u}$) via **GAT**.
- **Fusion:** Combines behavioral and social signals by simple summation:

$$h_u = h_{int} + h_{soc}$$

# Alternative Component: HCL (Failed Experiment)

**Idea:** Combine Heterogeneity (Social + Interest) with Contrastive Learning + LightGCN.

**Architecture Diagram:**




**Note: Contrastive Loss proved unstable in this sparse setting.**

# Table of Contents


1. Project Overview & Methodology
2. Dataset Construction and EDA
3. Motivation: Graph Expansion Strategy
4. State-of-the-Art (SOTA) Baselines
5. Proposed Method: Heterogeneous Graph Learning
6. Experimental Results & Innovations
7. Conclusion & Future Works

# Experimental Setup

**Models ($N = 6$):**

- **Baselines:** LightGCN, HGN
- **Contrastive:** LightGCL, SimGCL, XSimGCL (SOTA), HCL (Ours)

**Training config:**

- **Epochs:** 100
- **Batch Size:** 2048
- **Loss:** BPR + InfoNCE (for CL models)

**Evaluation Protocols:**

- **Full (Cold-Start):** 80/20 random user split.

**Graph Tiers:**

- **Strict G1:** Bipartite (User-Art)
- **Strict G2:** Hetero (+ Social)
- **Strict G3:** Hetero (+ Category)

# Evaluation Methodology & Validity

Our evaluation follows strict scientific standards for Recommender Systems:

- **Metric Choice:**
  - **Recall@10**: Measures system coverage.
  - **NDCG@10**: Rewards models that rank true items higher.
  - **MRR**: Standard for single-target news retrieval.
- **Implicit Feedback Validation:**
  - These metrics are standard for **Binary (Implicit)** interactions.
  - NDCG compensates for varying test set sizes across users.
- **Evaluation Strategy:**
  - **Full ranking (80/20):** Random user split. Train on 80% of users, test on 20% unseen users to measure zero-shot capability.

# Benchmark Results: Full Ranking

**Protocol:** 80/20 Split (Random).

**Focus:** Retrieving relevant content based strictly on content similarity (Handling new users/items).

| Model | Type | R@10 | N@10 | H@10 | MRR |
|---|---|---|---|---|---|
| ***Content-Based / Baselines*** | | | | | |
| **BGE-M3** | **CB** | **0.0535** | **0.0315** | **0.0929** | **0.0416** |
| TF-IDF | CB | 0.0508 | 0.0301 | 0.0951 | 0.0394 |
| User-KNN (Pearson) | Memory CF | 0.0412 | 0.0235 | 0.0762 | 0.0251 |
| ***Graph G1 (Bipartite)*** | | | | | |
| LightGCL | Graph CL | 0.0398 | 0.0249 | 0.0685 | 0.0313 |
| XSimGCL | Graph CL | **0.0439** | 0.0253 | 0.0694 | 0.0289 |
| HGN | Hetero GNN | 0.0262 | 0.0147 | 0.0451 | 0.0182 |
| HCL | Hetero CL | 0.0259 | 0.0164 | 0.0471 | 0.0214 |
| SimGCL | Graph CL | 0.0191 | 0.0115 | 0.0275 | 0.0135 |
| ***Graph G2 (Hetero - User-User)*** | | | | | |
| LightGCL | Graph CL | **0.0436** | 0.0261 | 0.0739 | 0.0313 |
| XSimGCL | Graph CL | 0.0432 | 0.0260 | 0.0688 | 0.0303 |
| HGN | Hetero GNN | 0.0369 | 0.0201 | 0.0596 | 0.0250 |
| HCL | Hetero CL | 0.0320 | 0.0192 | 0.0520 | 0.0233 |
| SimGCL | Graph CL | 0.0228 | 0.0131 | 0.0312 | 0.0144 |
| ***Graph G3 (Category Hubs)*** | | | | | |
| LightGCL | Graph CL | **0.0413** | 0.0256 | 0.0710 | 0.0311 |
| XSimGCL | Graph CL | 0.0379 | 0.0232 | 0.0643 | 0.0289 |
| HGN | Hetero GNN | 0.0299 | 0.0167 | 0.0464 | 0.0198 |
| HCL | Hetero CL | 0.0250 | 0.0152 | 0.0471 | 0.0205 |
| SimGCL | Graph CL | 0.0222 | 0.0122 | 0.0322 | 0.0137 |

# Impact of Graph Structure

**Question:** Which graph topology yields the best performance?
Testing **HGN** on different graph types:

| Graph | Structure Stats | R@10 |
|---|---|---|
| G1 (Base) | 13k Users, 30k Edges<br>Density: 0.06% | 0.0262 |
| **G2** | **+61k Social Edges**<br>Density: **0.064%** | **0.0369** |
| G3 | +6 Categories<br>16k User-Cat Edges | 0.0299 |

**Insight:** Denser is not always better. **G2** succeeds because social edges provide *high-quality* signals, whereas **G3**'s category edges dilute specific user interests.

# Efficiency Analysis: Model Complexity and Training Time

**Comparison:** Training time per epoch (minutes:seconds) and parameter count.

| Model | Params | Time (G1) | Time (G2) | Time (G3) |
|---|---|---|---|---|
| SimGCL | 1,072,768 | 00:47 | 00:46 | 00:45 |
| XSimGCL | 1,072,770 | **00:35** | **00:33** | **00:34** |
| LightGCL | 1,072,768 | 00:46 | 00:45 | 00:45 |
| MA-HCL | 1,072,770 | 00:47 | 01:10 | 00:37 |
| MA-HGN | 1,273,986 | 01:29 | 02:23 | 00:35 |

**Observations:**

- **XSimGCL** is the most efficient method (fastest training time).
- **MA-HCL** maintains comparable efficiency to SimGCL/LightGCL on bipartite graphs (G1, G3) while handling heterogeneity.
- **MA-HGN** incurs significant computational cost on dense heterogeneous graphs (G2), taking **2-3x longer** than Contrastive Learning methods.

# GNN Architecture

Comparing **GAT** (default), **GraphSAGE**, and **TransformerConv**.

| Backbone | Full Protocol | | Efficiency | | |
|---|---|---|---|---|---|
| | R@10 | N@10 i | Inf. (s) | Params | Train (s) |
| GAT | 0.0262 | 0.0163 | ∼1.38 | 1.27M | 20.23 |
| SAGE | 0.0303 | 0.0182 | ∼0.88 | 1.27M | 19.77 |
| **TRANS** | **0.0504** | **0.0266** | **1.71** | **1.47M** | 19.76 |

**Analysis:**

- **Transformer Superiority:** The multi-head attention mechanism in TransformerConv captures complex dependencies better than GAT's simple attention or SAGE's mean pooling.
- **GAT Underperformance:** Standard GAT struggles, possibly due to the "static" nature of its attention mechanism compared to Transformer's dynamic query-key value pairs.
- **Consistent Ranking:** TransformerConv dominates in Cold Start scenarios.

# Ablation Study: Impact of Meta-Paths

**Question:** How do different edge types contribute to performance?

| Configuration | Recall@10 | Drop (%) | Insight |
|---|---|---|---|
| **Full Graph (G2)** | **0.0369** | - | Best performance |
| w/o Social | 0.0293 | -20.6% | Social alignment is critical |
| w/o Category | 0.0227 | -38.5% | Categories provide broad signals |

**Conclusion:**

- **Synergy:** Combining all edge types (Social + Category + Behavior) yields the highest Recall.
- **Category Impact:** Removing categories hurts performance the most, indicating they are essential for semantic alignment in cold-start scenarios.

# Our Contributions: HGN & HCL

We propose two distinct architectures to handle the "VnExpress Heterogeneity":

## 1. HGN (Best for Structure)

- **Method:** Meta-path Aggregation + Attention Fusion.
- **Why:** Explicitly weighs "Social" vs "Interest".
- **Recall@10 (G2): 0.0369** (Highest Graph Model).

## 2. HCL (Contrastive)

- **Method:** Heterogeneity + Contrastive Loss.
- **Why:** Regularization against noise.
- **Recall@10 (G2):** 0.0320 (Stable).

**Key Takeaway:**

- **HGN** successfully exploits the Social graph (G2) to achieve the best structural performance (+40% over G1).
- **HCL** offers a lightweight alternative, though attention mechanisms

# Ablation Study: Impact of GNN Depth ($L$)

**Question:** Does deeper graph propagation help? **Metric:** Recall@10 on Full Ranking.

| Model | L=1 | L=2 | L=3 | L=4 |
|---|---|---|---|---|
| LightGCL | 0.032 | **0.040** | **0.043** | **0.042** |
| XSimGCL | 0.028 | 0.035 | 0.039 | 0.041 |
| LightGCN | **0.031** | 0.032 | 0.32 | 0.031 |
| HGN | **0.033** | 0.026 | 0.026 | 0.028 |
| HCL | 0.020 | 0.010 | 0.007 | 0.009 |
| SimGCL | 0.024 | **0.027** | 0.021 | 0.023 |

**Insight:**

- **LightGCN** suffers from *over-smoothing* (Performance drops after L=1).
- **XSimGCL** and **LightGCL** maintain robustness at deeper layers, capturing long-range dependencies effectively.

# Performance Justification: Challenge of Sparsity

**Is 3-4% Recall Low?** No. Recommender performance heavily depends on **Graph Density**.

| Dataset | #Users | #Items | Density | SOTA Recall@20 |
|---|---|---|---|---|
| ML-10M | 69k | 10k | **0.982%** | $\sim$ 0.2613 |
| Yelp | 29k | 24k | 0.146% | $\sim$ 0.0793 |
| Gowalla | 50k | 57k | 0.040% | $\sim$ 0.1578 |
| Amazon-Book | 78k | 77k | **0.037%** | **0.0585** |
| Tmall | 47k | 41k | 0.119% | 0.0528 |
| **Ours (G2)** | **13k** | **3.6k** | **0.08%** | **0.0369** (@10) |

Table 1: Comparison with Standard Benchmarks (LightGCN/SimGCL Results)

**Our Dataset vs. Amazon-Book:**

- Similar high sparsity ($\sim 0.0X\%$).
- News items have **shorter lifespans** than books/products, making "exact retrieval" (Recall) harder.

# Table of Contents


1. Project Overview & Methodology
2. Dataset Construction and EDA
3. Motivation: Graph Expansion Strategy
4. State-of-the-Art (SOTA) Baselines
5. Proposed Method: Heterogeneous Graph Learning
6. Experimental Results & Innovations
7. Conclusion & Future Works

# Conclusion

**Key Achievements:**

- Built an end-to-end **Heterogeneous Graph Recommendation System** for Vietnamese News.
- Integrated **Social & Temporal** signals, resolving the user cold-start and sparsity problem.

**Limitations:**

- **Poor Performance:** MA-HGN  MA-HCL yield bad results (Recall10 $\sim 3.7\%$).
- **Computational Cost:** Graph construction and neighborhood aggregation (GNN) are resource-intensive for large-scale graphs.
- **Offline Latency:** The current model is static; new articles require re-training or expensive incremental updates (no real-time adaptation).

# Future Directions

**Roadmap for Improvement:**

- **Dynamic Graph Learning (dyHGCN):** Capture evolving user interests over time instead of static snapshots.
- **More Meta Path:** Use more meta paths to capture different types of relationships between entities.
- **Self-Supervised Learning (SSL):** Apply contrastive learning to de-noise the graph and improve robustness against sparsity.
- **Knowledge Distillation:** Compress the large GNN model into a lightweight student model for faster inference.

# Thank You!

Q & A

# References I


[1] J. Shi, H. Ji, C. Shi, X. Wang, Z. Zhang, and J. Zhou, "Heterogeneous graph neural network for recommendation," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[2] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "Lightgcn: Simplifying and powering graph convolution network for recommendation,", 2020. arXiv: 2002.02126 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2002.02126